Bus crews: cover every trip at the lowest cost

A timetable of trips is fixed; we choose which driver does which trips, legally, as cheaply as possible.


Bus drivers


Airline crew pairing


Train crew rostering


Delivery drivers

Same structure everywhere: chains of tasks, legal limits, a cost per crew member

One driver is worth 10 hours of waiting

Inputs	Trips (start, end, from, to) between terminals
A duty	One driver's chain of trips: each leaves from where the last arrived, at least 10 min later
Rule 1: spread	First departure to last arrival ≤ L (e.g. 7 h)
Rule 2: driving	Total driving ≤ W = 0.75 L
Cost of a duty	$300 + $0.50 per minute of spread (waiting is paid)
Goal	Every trip covered exactly once, at minimum total cost

$300 ÷ $0.50/min

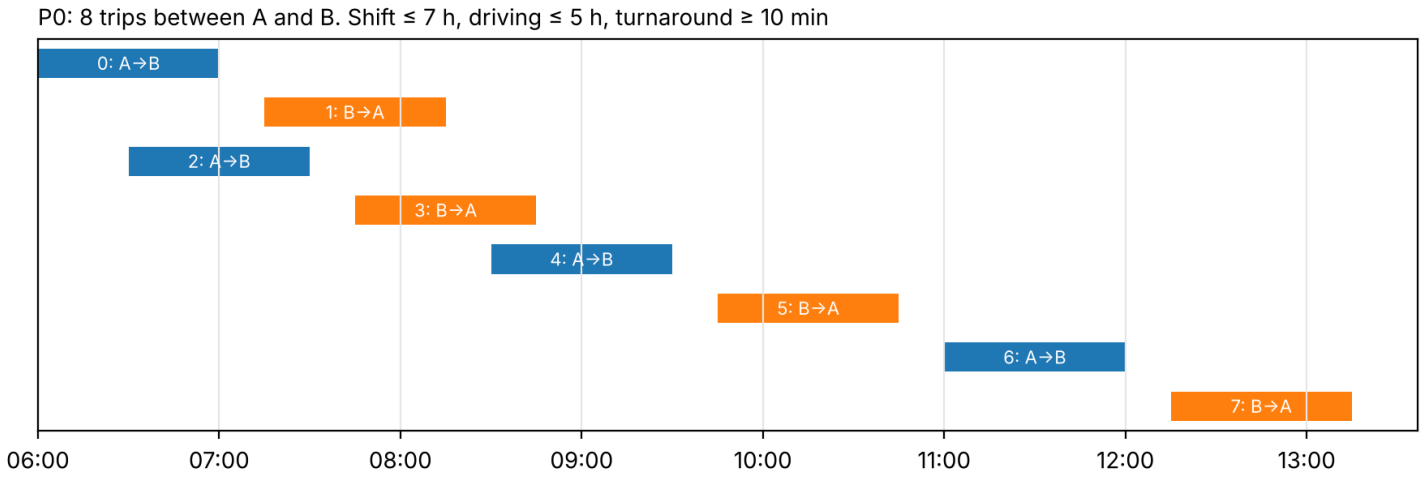
600 min

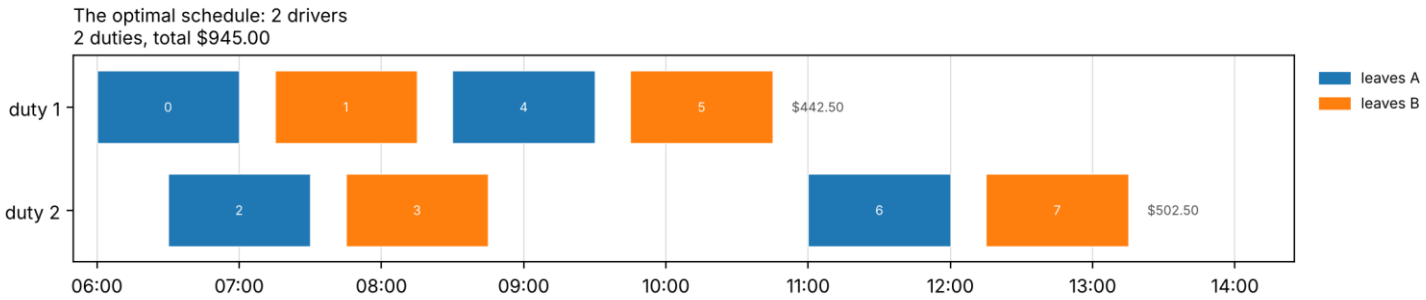
= 10 hours of paid waiting

So it is worth making drivers wait a lot to avoid hiring one more.

P0: can you find the 2-driver schedule? One driver is impossible

8 one-hour trips between A and B · shift ≤ 7 h · driving ≤ 5 h





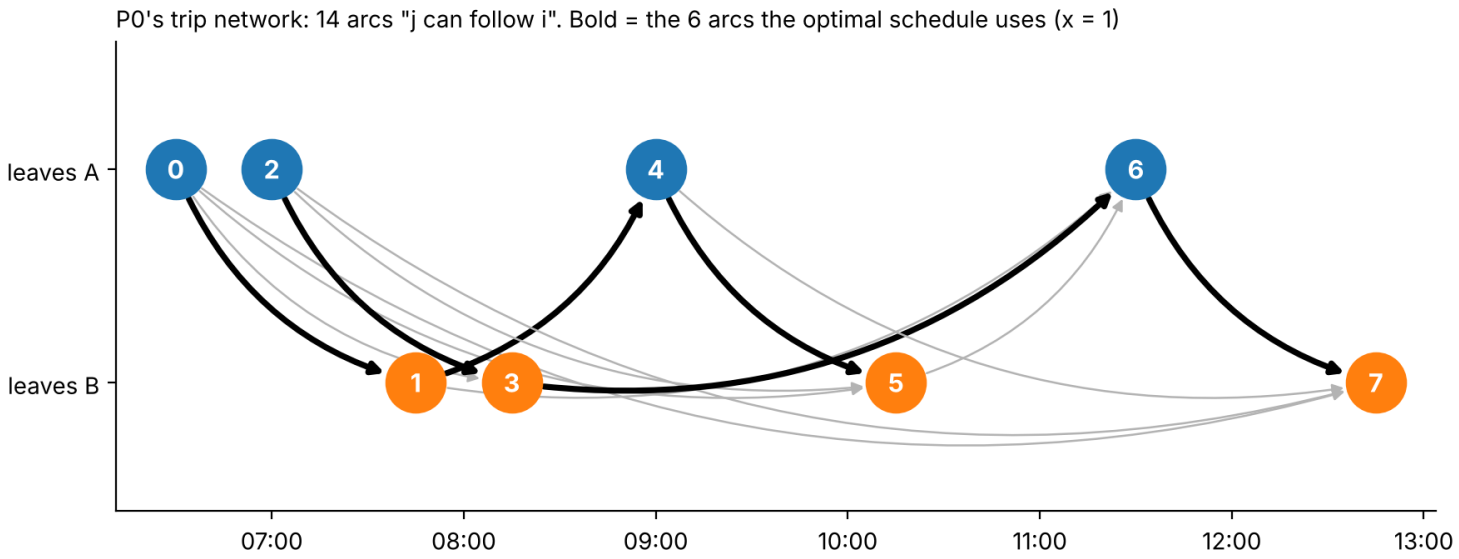
Why not 1 driver?

- Trips 0 and 2 overlap
- 8 h of driving > 5 h allowed

$945

2 drivers: {0, 1, 4, 5} $442.50 + {2, 3, 6, 7} $502.50, proven optimal

Model 1 writes a schedule as chains of trips (arc-flow)



Arrow i → j: j leaves where i arrives, ≥ 10 min later. Bold: the 6 arrows of the optimal schedule.

x — 1 if the same driver does j right after i (one per arrow)

u, v — 1 if a trip starts / ends a duty

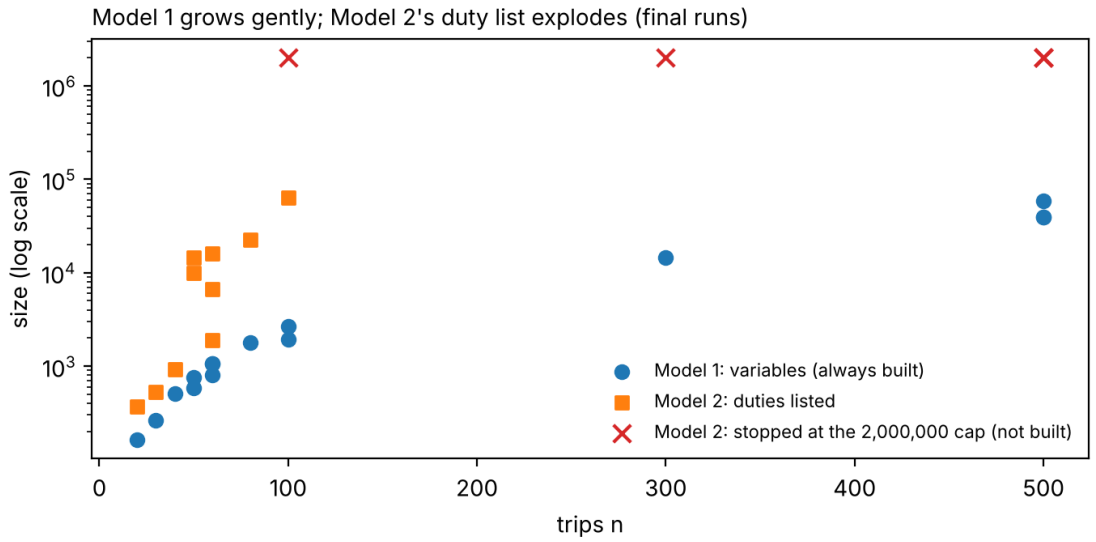
T — when the duty started (spread ≤ L)

D — driving so far (≤ W)

Every trip: one way in, one way out → the chosen arrows form chains, one chain per driver

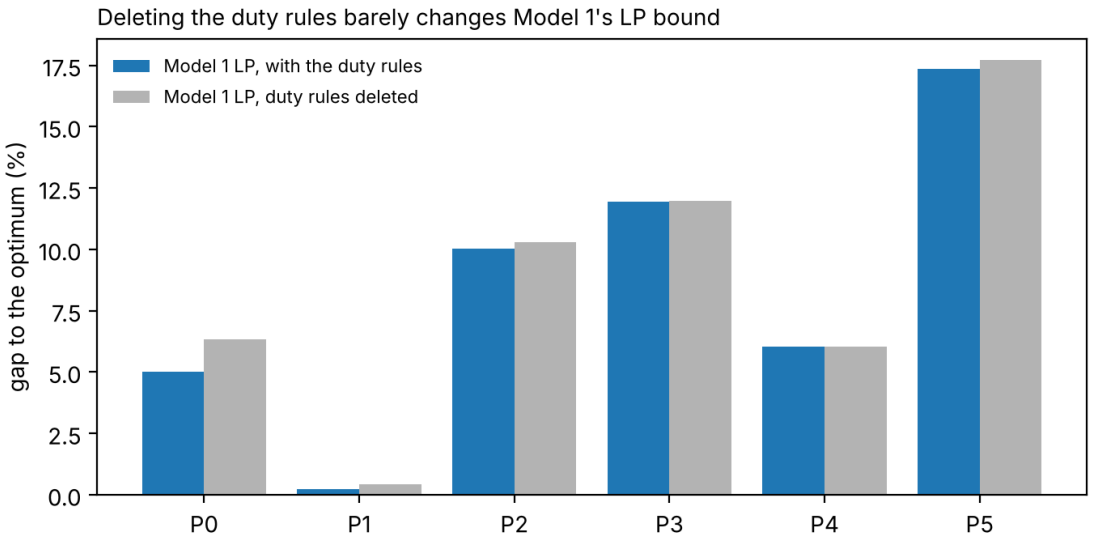
Model 1 is small and always buildable, but its LP almost forgets the duty limits

Strength: size



Under 60,000 variables at 500 trips; Model 2 not built on 5 of 15

Weakness: the LP bound



Deleting rules (4)-(7) barely moves it (P0-P5); not at all on P3, P4

The rules that make the problem hard only bite once the solver branches

Model 2 writes a schedule as a menu of whole working days

Example P0: 8 trips, 2 terminals

Trip	Route	Time
0	A → B	06:00–07:00
1	B → A	07:15–08:15
2	A → B	06:30–07:30
3	B → A	07:45–08:45
4	A → B	08:30–09:30
5	B → A	09:45–10:45
6	A → B	11:00–12:00
7	B → A	12:15–13:15

The menu: 6 of the 40 legal duties (● = duty covers the trip)

Trip	Duty 1	Duty 2	Duty 3	Duty 4	Duty 5	Duty 6
trip 0	●	●	●			
trip 1	●	●				
trip 2				●	●	
trip 3			●	●		
trip 4	●					
trip 5	●				●	
trip 6			●	●	●	
trip 7				●	●	●
cost	**$442.50**	**$367.50**	**$480**	**$502.50**	**$502.50**	**$330**

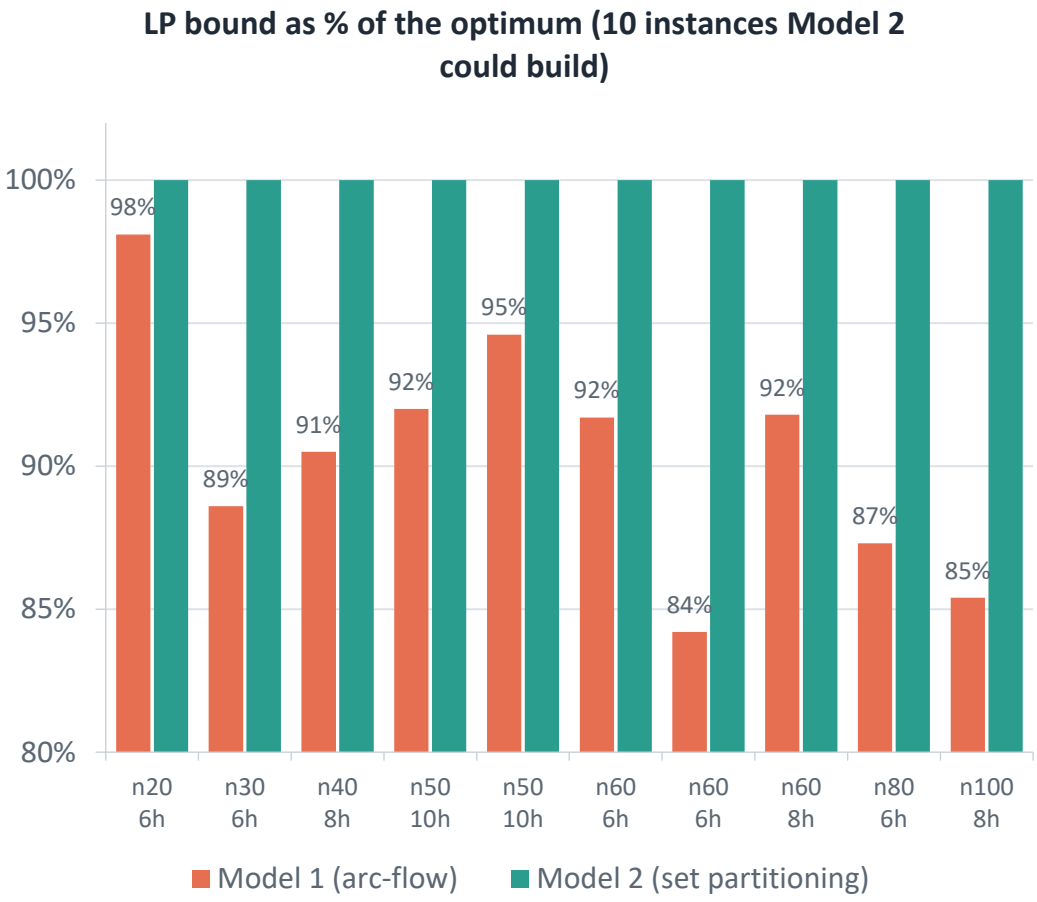
min Σ cost(p)·z(p) subject to **Σ** z(p) over duties containing trip i **= 1** for every trip i (8)

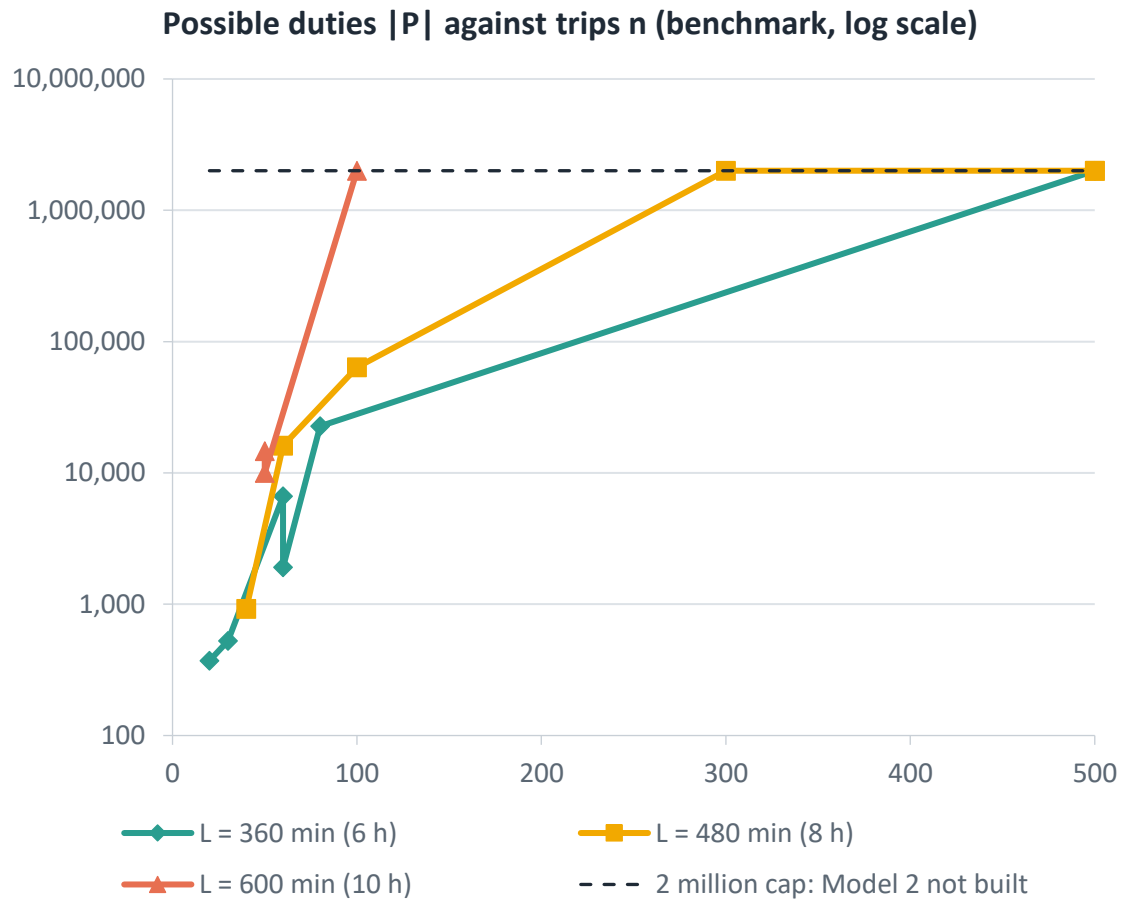
z(p) ∈ {0, 1}: 1 if duty p is used · all work rules are inside the menu, so no big-M

Gurobi picks duties 1 and 4:

2 drivers, $945

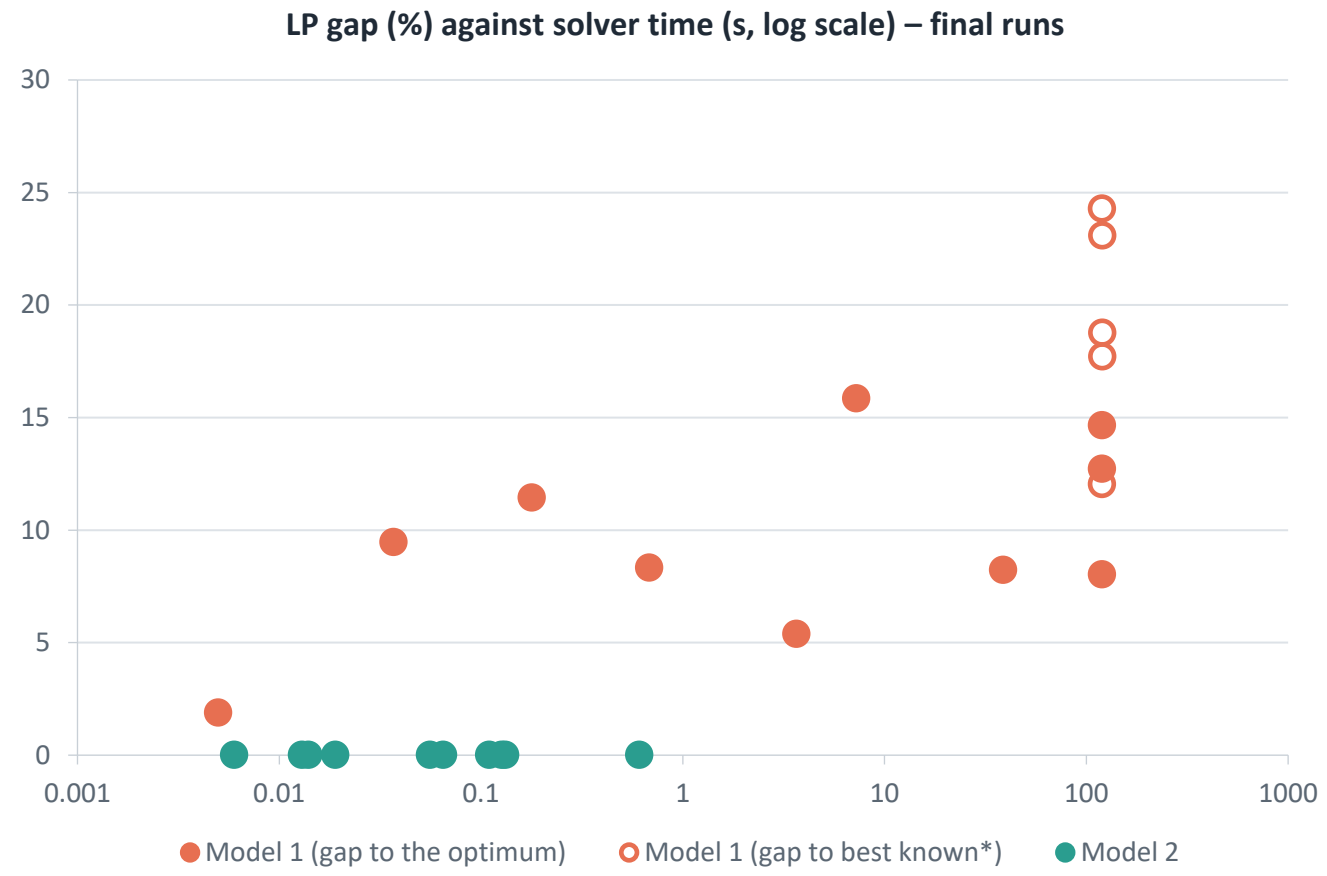
Model 2 has the stronger bound, but its menu explodes with longer duties





Model 2's LP bound equals the optimum on all 10 instances it could build; Model 1's is 84–98 % of it. Points on the cap: Model 2 not built.

Same optimum – and the stronger bound wins, until the menu cannot be built



10 of 10

Model 2 models that could be built: proven optimal at the root node, at most 0.9 s end to end

5 of 15

never built: 3–75 s spent listing duties, then the menu passed 2 million

* No optimum is known where Model 2 was not built: there the gap is measured to the best schedule found, which overstates it.

Benchmark: realistic bus timetables, from instant to impossible, chosen by a written rule

Every model is fast

4

both models optimal in under 1 s

The models differ clearly

6

Model 1 takes 3.5 s to the 120 s limit; Model 2 always under 0.6 s

No model finishes

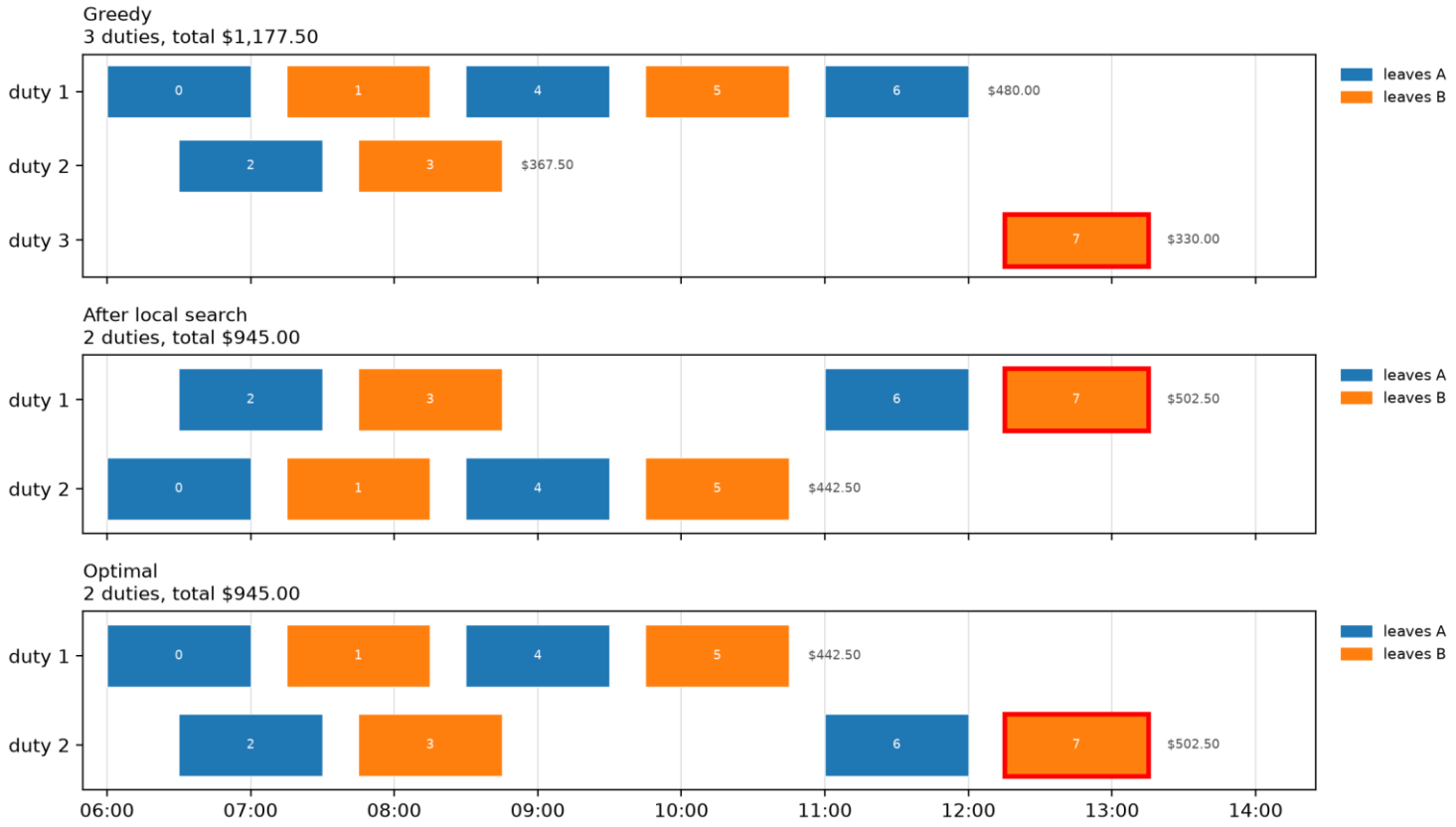
5

Model 1 times out; Model 2 cannot be built (> 2 million duties)

Class	Inst.	Trips n	Terminals	Duty length	Duties |P|	Model 1 (s)	Model 2 (s)
Easy	4	20–60	2–3	6–8 h	371–6,650	0.0–0.7	0.01–0.05
Medium	2	50–60	3	6–10 h	1,911–9,982	3.5–6.7	0.02–0.06
Hard	1	60	2	8 h	16,064	38.6	0.12
Time limit	8	50–500	2–3	6–10 h	14,599–63,633 · > 2 M	limit	0.11–0.60 · 5 not built

Chosen by a written rule fixed before the final runs (issue #1): spread across sizes · ≥ 2 where Model 2 cannot be built · ≥ 15 in total (15 of 43 candidates; 2 by the fill-up). Limitation: W = 0.75 L ties the two duty limits.

Greedy strands trip 7; swap-then-merge saves a whole driver ($232.50)



What the move pair is worth (P0)

	Greedy	After LS	Change
Drivers	3 × $300	2 × $300	**−$300**
Paid time	$277.50	$345.00	**+$67.50**
Total	**$1,177.50**	**$945.00**	**−$232.50**

1 driver = $300 = 10 h of paid waiting

Move trip 6 first (costs more alone), then merge trip 7: one driver fewer.

On the benchmark: a median 2.4 % above the optimum.

The heuristic gives both models a complete start; it rarely changes the result

Timetable
up to 500 trips

→

Greedy
shortest wait first

→

Local search
swap-then-merge

→

Complete start
Model 1: x, u, v, T, D
Model 2: z

→

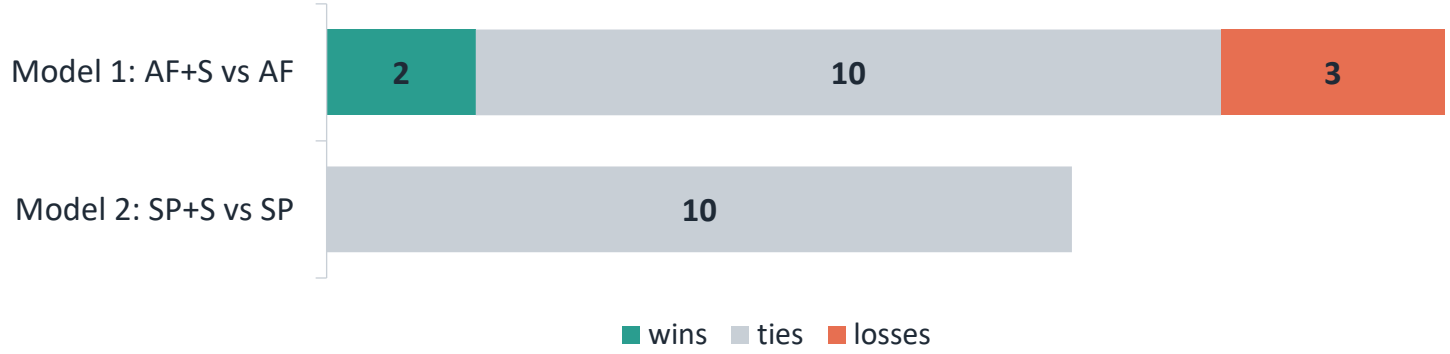
Gurobi
“User MIP start produced solution”

≤ 0.43 s
heuristic time, up to 500 trips

2.4 %
median above the optimum

25 / 25
starts accepted by Gurobi

Did the start change the result? (instances, B12 tie rule)



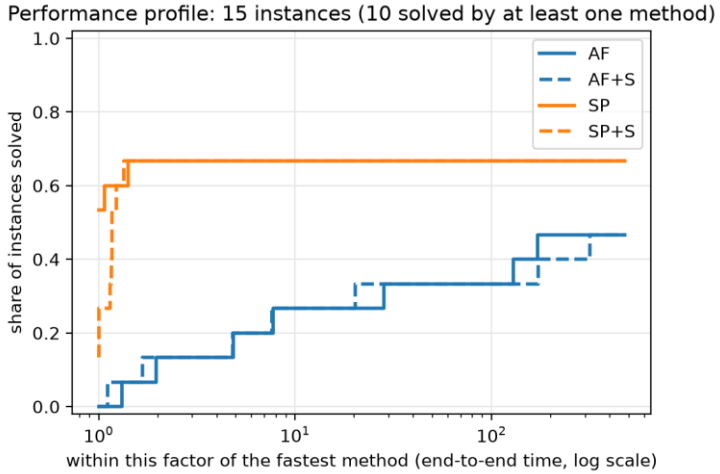
Wins: n = 500 ended 3.9 % cheaper; an n = 50 proof was 29 % faster.

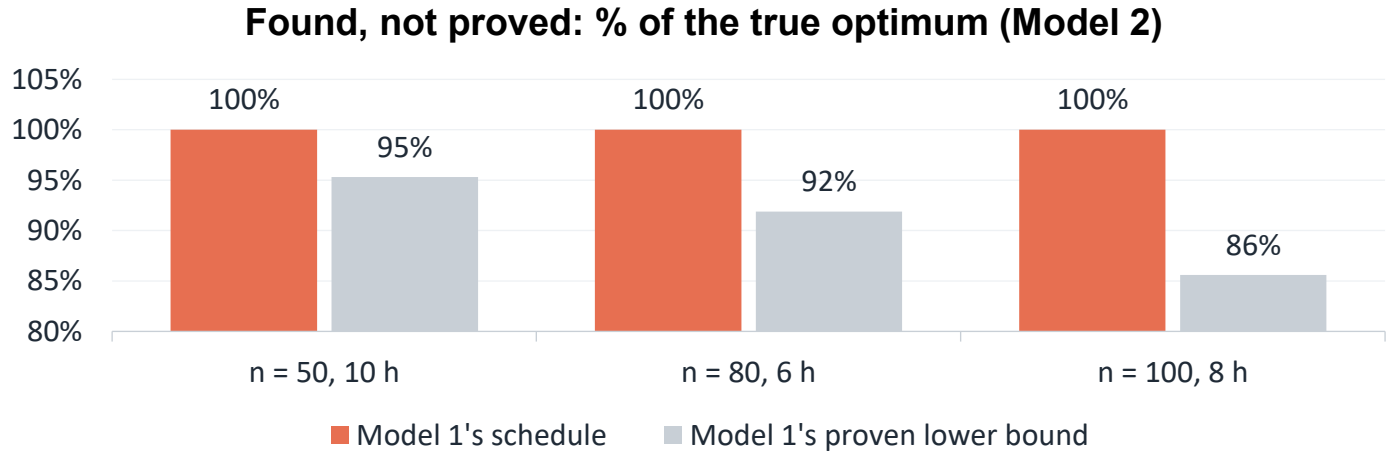
Losses: two quick proofs got slower, one bound got weaker — a different start, a different search tree.

Model 2 wins whenever it can be built; Model 1 is the only option beyond that

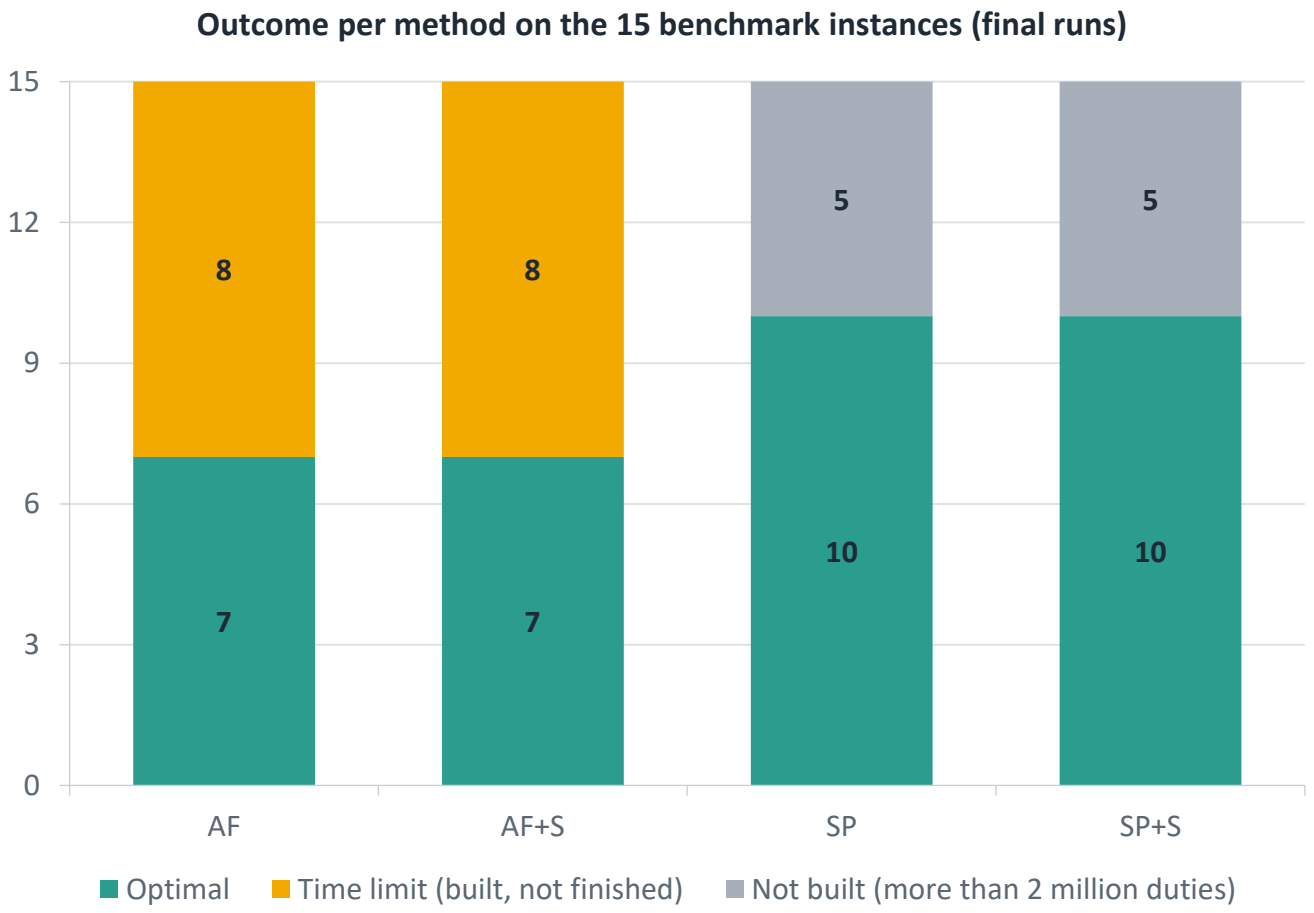
Method (15 instances)	Optimal	Time limit	Not built	Avg gap %	Max gap %	Avg solver s	Avg end-to-end s	Avg nodes
Heuristic	0	0	0	–	–	–	0.07	–
Model 1	7	8	0	8.49	27.19	≥ 67.42	≥ 67.84	≥ 61,910.13
Model 1 + start	7	8	0	8.25	24.27	≥ 69.68	≥ 70.19	≥ 46,000.60
Model 2	**10**	0	5	**0.00**	0.00	0.12	12.30	1.00
Model 2 + start	**10**	0	5	**0.00**	0.00	0.12	12.25	1.00

≥ = includes runs stopped at the 120 s limit (a lower bound). Solver time, gap and nodes: built runs only; end-to-end includes Model 2's failed builds (3-75 s each).





Two different failures: Model 1 runs out of time from 80 trips, Model 2's menu stops fitting from 100



Model 1: 8 of 15 time-outs

always built; every instance from n = 80 (and one n = 50) stops at 120 s with a 5–27 % gap

Model 2: 5 of 15 not built

optimal on all 10 it builds, at the root node; never built at n = 100 with 10 h duties or at n ≥ 300

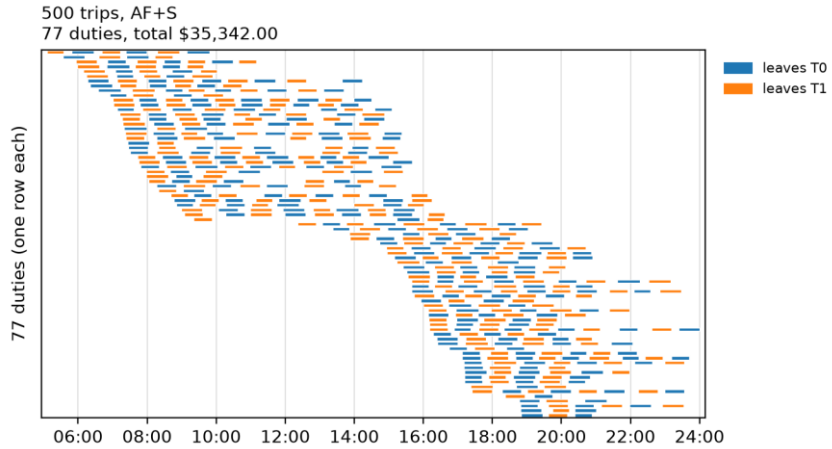
Use the menu while it fits.

Next step: column generation – build only the duties worth adding.

Theory held for the bounds and the cliff; the start and the build share surprised us

	We expected	Verdict	Evidence
H1	Model 1's LP ≤ Model 2's LP	**confirmed**	10 / 10; Model 2's LP = optimum
H2	Naive big-M is a little weaker	**confirmed: no gap**	identical on 15 / 15
H3	Model 2: fewer nodes, faster	**confirmed**	faster on 6 / 7, 1 node each
H4	Model 2's build share grows	**not confirmed**	falls 72 % → 32 %
H5	Model 2 becomes unbuildable	**confirmed**	5 / 15 not built; Model 1: 0
H6	The start helps Model 1 more	**not confirmed**	median 2.6 % vs 2.6 %
H8	Trips per duty matter more than n	**cannot be decided**	rank corr. 0.84 vs 0.83

H7 (reduced-cost fixing) was withdrawn in plan v1.5; H9 and H10 were optional and not run.



Largest instance: 500 trips, 77 drivers, $35,342

Model 1 with the start, proven gap 23.1 %; every duty re-checked by the independent checker.

Limits: one generator family, one machine, one seed per configuration; the driving limit is tied to the spread.

Next: column generation — generate only duties that can improve the solution (a shortest path with two resources), so Model 2 keeps its bound without listing millions of duties.